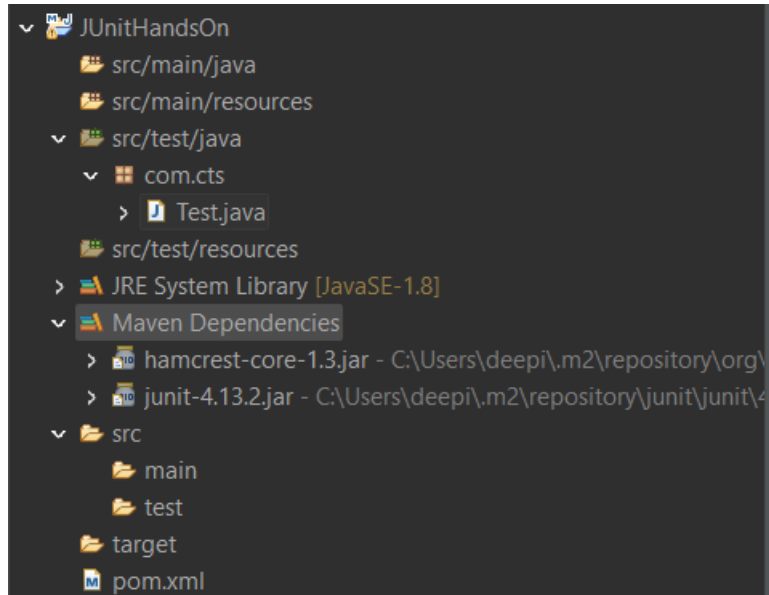
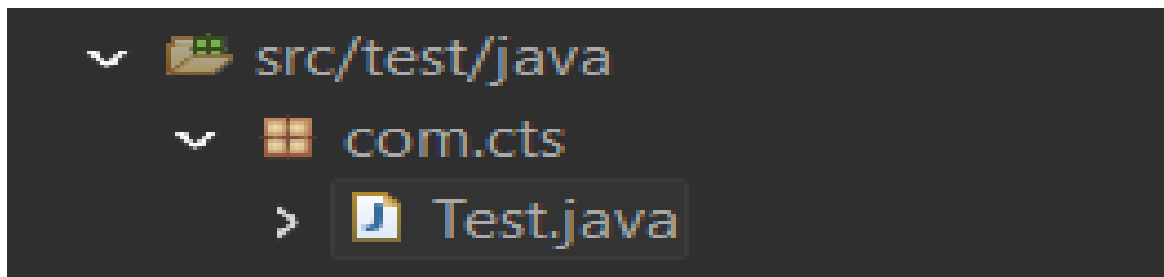


JUnit Testing Exercises

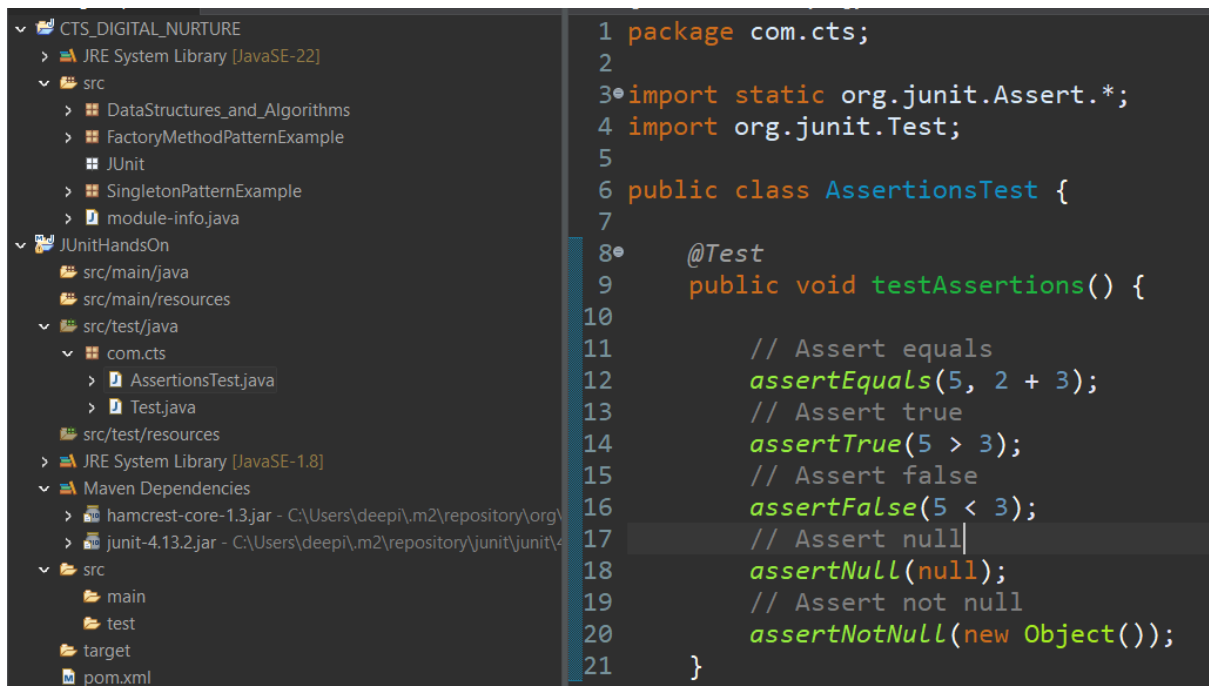
Exercise 1: Setting Up JUnit

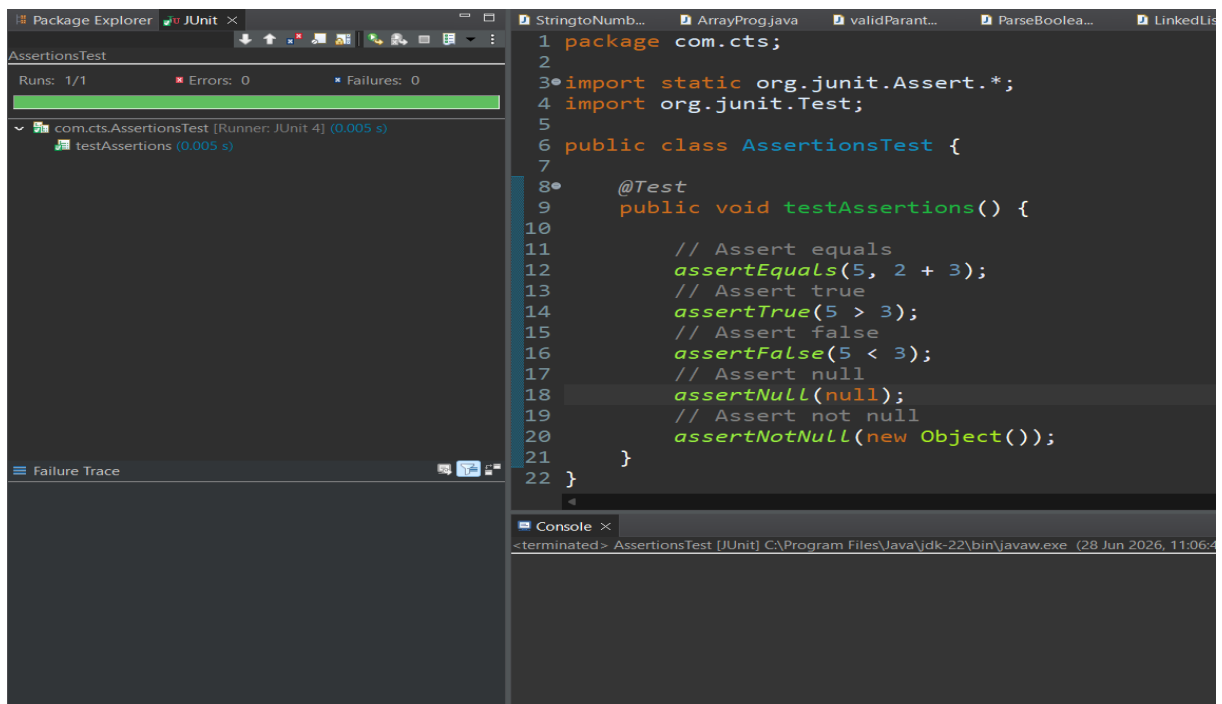
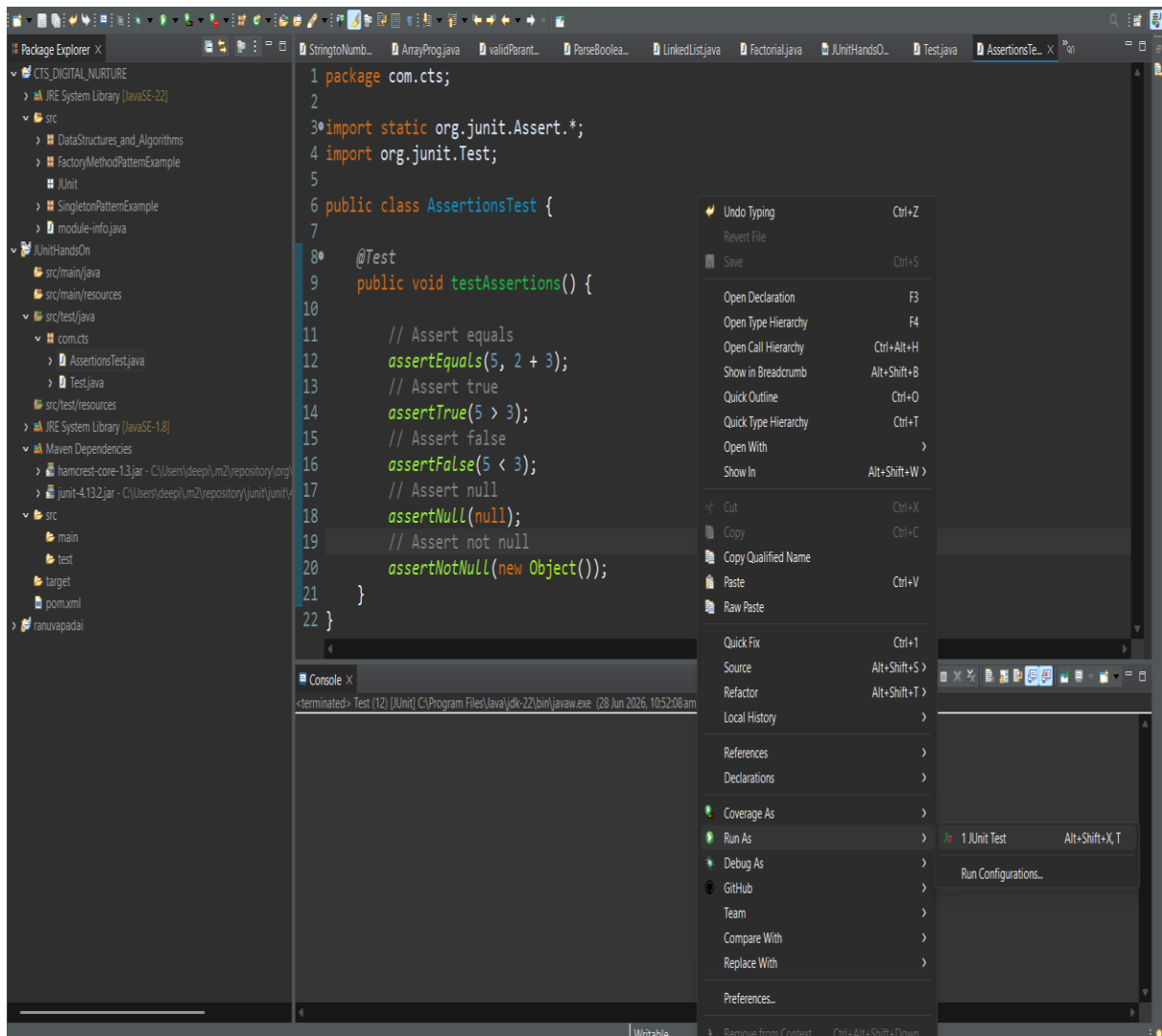


```
1<project xmlns="http://maven.apache.org/POM/4.0.0"
2      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4
5    <modelVersion>4.0.0</modelVersion>
6
7    <groupId>com.cts</groupId>
8    <artifactId>JUnitHandsOn</artifactId>
9    <version>0.0.1-SNAPSHOT</version>
10   <name>JUnitHandsOn</name>
11
12   <dependencies>
13     <dependency>
14       <groupId>junit</groupId>
15       <artifactId>junit</artifactId>
16       <version>4.13.2</version>
17       <scope>test</scope>
18     </dependency>
19   </dependencies>
20
21 </project>
```



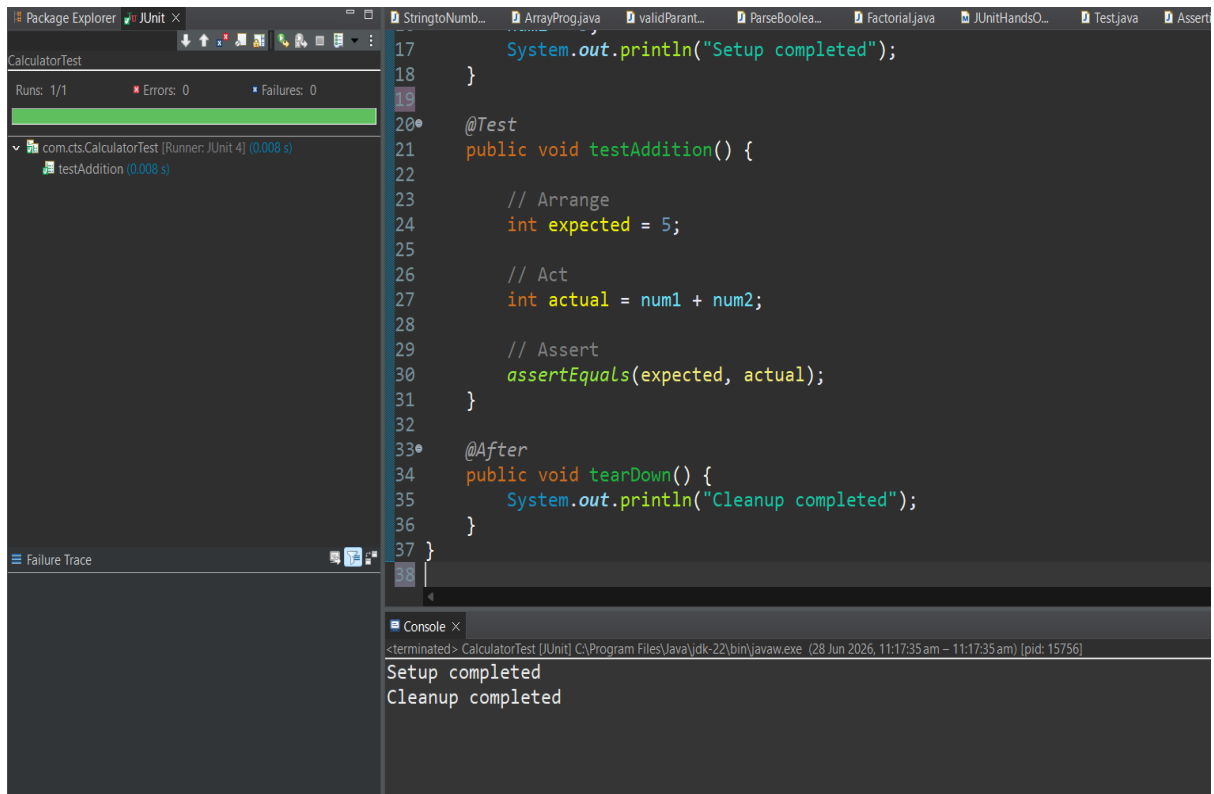
Exercise 3: Assertions in Junit





Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in Junit

```
1 package com.cts;
2
3 import static org.junit.Assert.*;
4 import org.junit.After;
5 import org.junit.Before;
6 import org.junit.Test;
7
8 public class CalculatorTest {
9
10     private int num1;
11     private int num2;
12
13     @Before
14     public void setUp() {
15         num1 = 2;
16         num2 = 3;
17         System.out.println("Setup completed");
18     }
19
20     @Test
21     public void testAddition() {
22         // Arrange
23         int expected = 5;
24
25         // Act
26         int actual = num1 + num2;
27
28         // Assert
29         assertEquals(expected, actual);
30     }
31
32     @After
33     public void tearDown() {
34         System.out.println("Cleanup completed");
35     }
36 }
37 }
```



Mockito Hands-On Exercises

Exercise 1: Mocking and Stubbing



```
1 package com.cts;
2 public interface ExternalApi {
3
4     String getData();
5
6 }
7
```

```
1 package com.cts;
2
3 public class MyService {
4
5     private ExternalApi api;
6
7     public MyService(ExternalApi api) {
8         this.api = api;
9     }
10
11     public String fetchData() {
12         return api.getData();
13     }
14 }

```

```
3 import static org.junit.Assert.assertEquals;
4 import static org.mockito.Mockito.when;
5
6 import org.junit.Test;
7 import org.mockito.Mockito;
8
9 public class MyServiceTest {
10
11     @Test
12     public void testExternalApi() {
13
14         ExternalApi mockApi = Mockito.mock(ExternalApi.class);
15
16         when(mockApi.getData()).thenReturn("Mock Data");
17
18         MyService service = new MyService(mockApi);
19
20         String result = service.fetchData();
21
22         assertEquals("Mock Data", result);
23     }
24 }

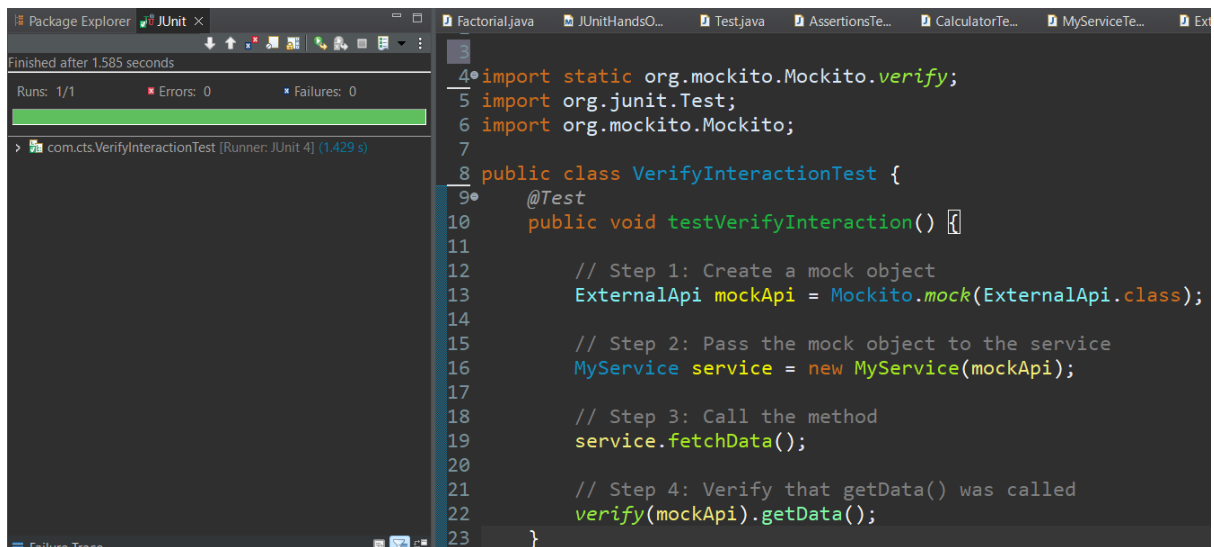
```

```
2
3 import static org.junit.Assert.assertEquals;
4 import static org.mockito.Mockito.when;
5
6 import org.junit.Test;
7 import org.mockito.Mockito;
8
9 public class MyServiceTest {
10
11     @Test
12     public void testExternalApi() {
13
14         ExternalApi mockApi = Mockito.mock(ExternalApi.class);
15
16         when(mockApi.getData()).thenReturn("Mock Data");
17
18         MyService service = new MyService(mockApi);
19
20         String result = service.fetchData();
21
22         assertEquals("Mock Data", result);
23     }
24 }

```

Exercise 2: Verifying Interactions

```
4•import static org.mockito.Mockito.verify;
5 import org.junit.Test;
6 import org.mockito.Mockito;
7
8 public class VerifyInteractionTest {
9     @Test
10     public void testVerifyInteraction() {
11
12         // Step 1: Create a mock object
13         ExternalApi mockApi = Mockito.mock(ExternalApi.class);
14
15         // Step 2: Pass the mock object to the service
16         MyService service = new MyService(mockApi);
17
18         // Step 3: Call the method
19         service.fetchData();
20
21         // Step 4: Verify that getData() was called
22         verify(mockApi).getData();
23     }
24 }
```



Logging using SLF4J

Exercise 1: Logging Error Messages and Warning Levels

